Balanced Bagging Classification on Census-Income (KDD)

MarcAnthony Cantu

May 2026

Describing the dataset

This project aims to explore techniques of training a classification model on a dataset in which the binary classifications are imbalanced.
That is to say, given a dataset with 2 classes CL_{min}, and CL_{maj} our imbalance ratio is somewhere in the range of:

$$0.02 \leq \frac{CL_{min}}{CL_{maj}} \leq 0.12$$

The dataset I am using is the **Census-Income dataset** which contains survey data on US citizens during 1994 to 1995. Each case represents answers given by a participant on a multitude of questions. The task is to classify, based on the participants answers, whether their annual income is above or below 50k.

$$CL_1 :< 50k \ , \ CL_2 :\geq 50k$$

Among the 40 features some examples are:

Feature	type	size (if categorical)
Age	Numerical	n/a
Education level	categorical	17
Occupation	categorical	47
State of previous residence	categorical	50

Table 1: Caption

The size of our classes are

$$CL_{min} = \{\text{income} \geq 50\text{k}\} = 12382,$$
$$CL_{maj} = \{\text{income} < 50\text{k}\} = 187141$$

With 95130 total cases and an imbalance ratio of

$$\frac{CL_{min}}{CL_{maj}} = 0.06$$

Formatting the data

Due to the large size of many categorical features, applying one-hot encoding will yield roughly 450 features. This will create complications with computation times and memory access due to requiring a large number of trees.
For this reason, an **ordinal encoding** was imposed on the categorical features which comes with its own issues since most of these features are not order-able in a natural way.

After applying ordinal encoding to the categorical features, we create a validation set V which is composed of 10% from CL_{min} and 10% from CL_{maj}. We denote the remainders of CL_{min} and CL_{maj} as R_{min} and R_{maj} respectively and

$$RD = R_{min} \cup R_{maj}$$

Balanced Bagging

To preform balanced bagging we utilize the `BalancedBaggingClassifier()` function from the `imbalanced-learn` python library with the following parameters:

```
    B = 51
    bbc = BalancedBaggingClassifier(
    estimator=RandomForestClassifier(n_jobs=-1),
    n_estimators=B, # bagging size
    bootstrap=False, # no extra resampling of RminCL
    sampling_strategy='majority',
    random_state=42
)
```

In order to evaluate each individual trees accuracy percentages we constructed the confusion matrix for each tree when tested on V. The diagonals of this confusion matrix gives us accuracy percentages p_{min} and p_{min} each a vector of size B

```
test_pmin = []
test_pmaj = []
for estimator in bbc.estimators_:
    y_pred = estimator.predict(X_valid)
    cm = confusion_matrix(y_valid, y_pred, labels=[1,0], normalize='true')
    test_pmaj.append(cm[0, 0])
    test_pmin.append(cm[1, 1])
```

We also evaluate the entire balanced bagging models accuracy percentages through its confusion matrix on V and on the training dataset RD with values:

Percentage accuracy on validation set	
$test_bag_p_{min}$	88.69%
$test_bag_p_{maj}$	84.11%
Percentage accuracy on whole dataset	
$train_bag_p_{min}$	99.98%
$train_bag_p_{maj}$	85.43%

The difference in accuracies for the minority and majority classes are 11.29% and 1.32% respectively. The small change in the accuracy of the majority class gives good indication that no substantial overfitting is occurring.

Results Analysis

A scatter plot was generated to display the planar points of all $[p_{min}, p_{maj}]$ as well as the balanced bagging percentage accuracy $[P_{min}, P_{maj}]$:

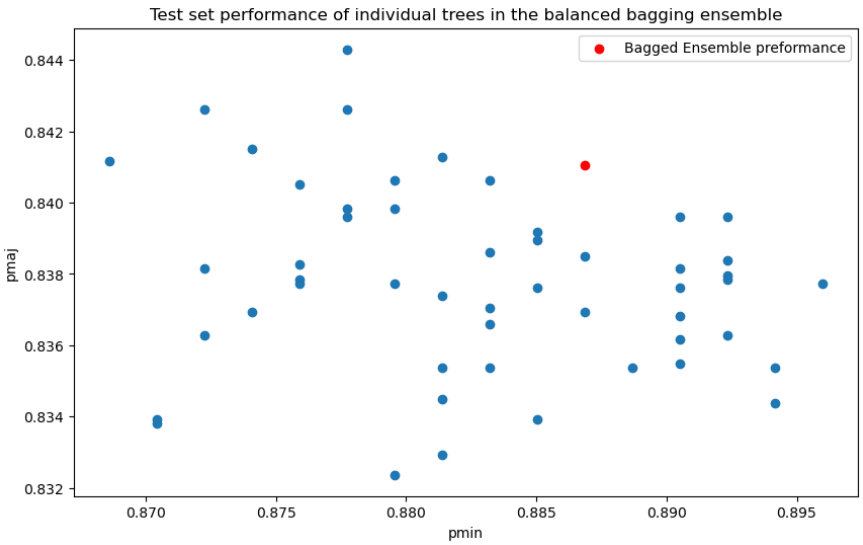


Figure 1

From the graph we can see that the percentage accuracies of all the individual trees lie in upper right quadrant indicating decent preformance on both the majority class and minority class.

Hyper-parameterization

We now aim to optimize out balanced bagging model with respect to the minority class by hyper-parameterization. The parameters we will iterate over are:

Parameter	values
`n_estimator`	$[100, 150, 200, 300]$
`min_sample_leaf`	$[1, 5, 10, 15, 20]$
`max_depth`	$[None, 10, 20, 30, 50, 75, 100]$
bagging size	$[51, 101, 201, 301]$

After iterating through all parameter values we ended with a new percentage accuracy of

$$P^*_{min} = 90.69\%$$
$$P^*_{maj} = 81.04\%$$

Raising out minority accuracy by 2% but dropping our majority accuracy by 3.07%. Computation times for each training session are given with best performing value (in accuracy) being highlighted:

n_estimators	time (seconds)
100	30.16s
150	38.69s
200	55.73s
300	101.10s
min_samples_leaf	**time (seconds)**
1	32.18s
5	12.98
10	7.96s
15	6.92
20	6.20s
max_depth	**time (seconds)**
None	6.99s
10	5.74s
20	6.17s
30	6.90s
50	6.74s
75	7.20s
100	7.22s
Bagging size	**time (seconds)**
51	5.65s
101	10.59s
201	21.12s
301	32.11s

After all is said and done the final parameter values chosen were:

Parameter	values
n_estimator	100
min_sample_leaf	15
max_depth	10
bagging size	51

Bagging size $B = 51$ was chosen over $B = 301$ because of minuscule performance increase vs faster computation times.

The trade-off in optimizing for the minority class resulted in a greater loss of accuracy for the majority class compared to the gain of accuracy for the minority class. Due to the ordering imposed on the categorical features, compilations with optimization are likely to arise. However, the computation times are substantially smaller than they would be had we imposed one-hot encoding.

Balanced Random Forest Classifier

We now move on to an alternative model known as **Balanced Random Forest Classifier** Here is a description of each hyperparameter for the BRF fucntion:

Main hyperparameters

The principal hyperparameters of `BalancedRandomForestClassifier` and their roles are:

`n_estimators` (default 100) — Number of trees in the forest.

`max_depth` (default `None`) — Maximum depth of each tree. `None` grows trees until leaves are pure or contain fewer than `min_samples_split` samples.

`min_samples_split` (default 2) — Minimum samples required to split an internal node. Larger values produce shallower, more regularized trees.

`min_samples_leaf` (default 1) — Minimum samples required at a leaf node. Increasing it forces smoother decision boundaries and reduces overfitting.

`max_features` (default `'sqrt'`) — Number of features considered at each split. The $\sqrt{F}$ default is the standard Breiman setting that drives the decorrelation between trees.

`bootstrap` (default `True`) — Whether the input data is bootstrapped before the per-tree undersampling step.

`sampling_strategy` (default `'auto'`) — Controls which classes are undersampled. The settings `'auto'` and `'majority'` both reduce the majority class to the minority size in the binary case.

`replacement` (default `False`) — Whether the per-tree undersampling is performed with or without replacement.

`class_weight` (default `None`) — Optional reweighting of classes in the impurity criterion, orthogonal to the sampling-based balancing.

`random_state`, `n_jobs` — Seed for reproducibility and CPU-parallelism control.

Algorithmic differences: BBC vs BRF

Both BBC and BRF combine bagging with class balancing, but they differ in where the balancing step occurs and in how trees are organized:

- **BBC.** Trains B independent base classifiers $\{\text{predW}_1, \ldots, \text{predW}_B\}$. Each base classifier is itself a fully grown Random Forest of `n_estimators` trees fit on a balanced subset

$$\text{TRA}_i \;=\; R_{min} \cup S_i, \qquad S_i \subset R_{maj}, \quad |S_i| = |R_{min}|.$$

  Each S_i is drawn randomly for bag i. The final bagged predictor is a vote across the B forests. Balancing happens at the *forest* level. Total tree count: $B \times$ `n_estimators`.

- **BRF.** Trains a single Random Forest of `n_estimators` trees. Each tree is grown on its own bootstrap of the input data, which is then random-under-sampled to balance the classes before fitting. Balancing happens at the *tree* level. Total tree count: `n_estimators`.

Applying BRF to dataset

We fit a BRF model to the *RD* dataset with the optimized parameters obtained in **T5** and evaluate both the hard voting performance and soft voting performance

Hard voting performance

Viewing the confustion matrix of the BRF on the validation set we get percentage accuracies of

$$BRF_p_{min} = 90.33\%$$
$$BRF_p_{maj} = 81.60\%$$

giving a slight gain of 0.52% accuracy for the majoirty class and a slight loss of 0.36% of accuracy on the minority class

Soft voting performance

To evaluate the soft voting performance of the BRF and BBC we display the ROC curves and AUC values for each:

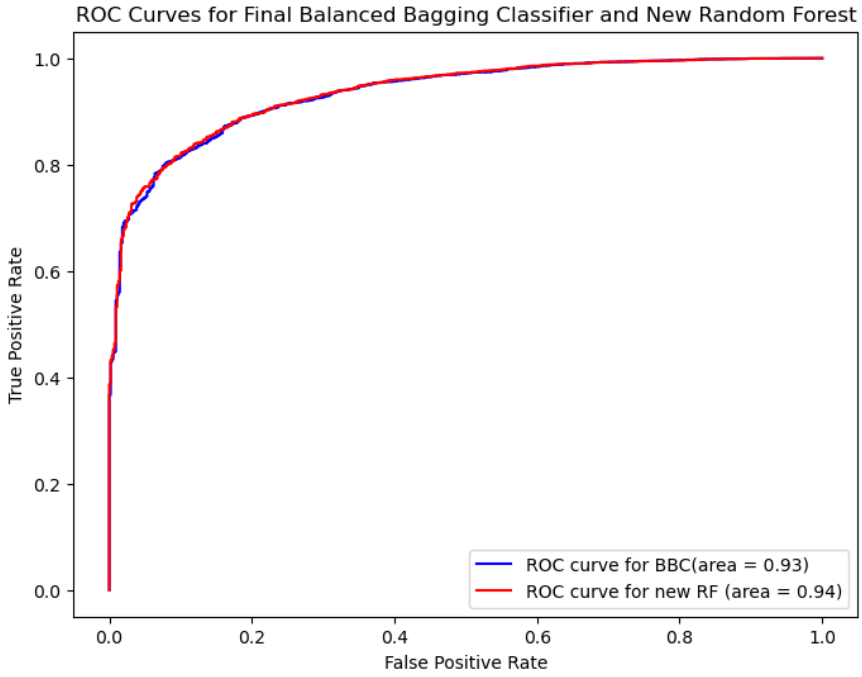


Figure 2

The ROC curves and AUC values gives indication that the BRF does slightly better than the BBC in the context of soft voting performance.

SMOTENC

We now attempt to improve the BRF by fitting it to a resampled dataset RD^* by utilizing the `SMOTENC` function in scikit-learn. This function is a variation on the `SMOTE` function that specialized in data that contains both numerical and categorical features and works by oversampling the minority class using **K-nearest neighbor interpolation**.
The parameters used for the `SMOTENC` function are:

```
    resampler = SMOTENC(
    categorical_features=cat_features, # column names of nominal features
    sampling_strategy=0.5, #imbalance ratio after resampling
    k_neighbors=10,
    random_state=42,
)
```

This will yield us a new dataset RD^* with an imbalance ratio of

$$\frac{R_{min}}{R_{maj}} = 0.5$$

After oversampling is done, we fit the BRF model to RD^* and evaluate the hard and soft voting performances:

$$BRF^*_p_{min} = 78.28\%$$
$$BRF^*_p_{maj} = 87.58\%$$

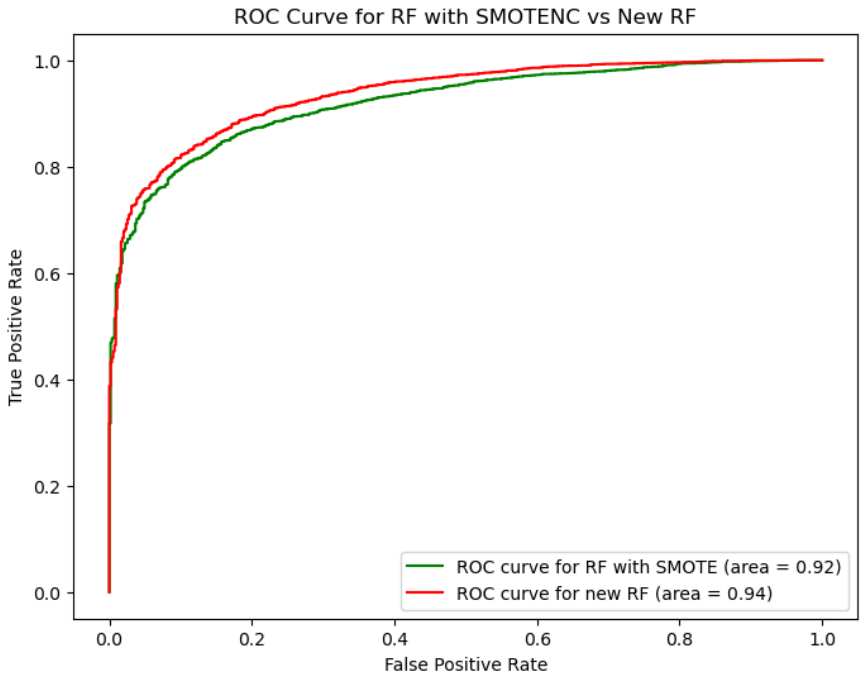


Figure 3

In regards to hard voting, the oversampled model BRF^* performed significantly worse than the original BRF in regards to the minority class with a difference of 12.05% and gained a comparatively small gain in performance on the majority class with a difference of 5.98%.

In regards to small voting, the oversampled BRF^* model is still visibly underperformed in comparison to the original BRF model.
These were unexpected results for me until I considered the fact that the balanced Random Forest Classifier is constructed to handle **imbalanced data**. Thus when we attempt to level out the class distribution the BRF is training on a synthetic population (imbalance ratio is 0.5 vs 0.06) while we test it in the true population resulting in a training-test distribution mismatch.